

Status	Method	Name	URL (Edge Function)	Input	Output	Flow / Reason	Notes
ACTIVE	POST	Fetch Weather	supabase. co/functions/v1/weather	{ lat: number, // rounded to 0.1 client-side lon: number }	{ temp: number, feels_like: number, humidity: number, // 0-100 condition: string, // e.g. "Clear" icon: string, // OWM icon code city: string, precipitation: number, // mm airQuality: string // "Good", "Moderate", "Bad", "Very Bad" }	useDataStore.fetchWeather() -> Edge Function weather/index.ts -> OpenWeatherMap current weather API -> OpenWeatherMap Air Pollution API (parallel)	Auth: no auth required (secrets server-side). Coordinates rounded to 0.1 precision client-side for cache reuse. AQI mapping: 1=Good, 2=Moderate, 4=Bad, 5=Very Bad. Timeout: 25s. Cache key: weather_(rLat)_(rLon) (6h TTL)
ACTIVE	POST	Fetch Stocks	supabase. co/functions/v1/stocks	{ symbols: string[] // e.g. ["KOSPI", "NASDAQ", "SP500", "USDKRW", "AAPL"] }	{ { symbol: string, price: number, change: number, // absolute changePercent: number // percent type: string currency: string }, ... }	useDataStore.fetchStocks(symbols) -> Edge Function stocks/index.ts -> Yahoo Finance (primary). Strict validation: price > 0 required. type and currency metadata auto-detected from Yahoo response.	Auth: no auth required. Symbols normalized to uppercase, 8 fixed index symbols. SP500->"GSPC", KOSPI->"KS11", NASDAQ->"IXIC", USDKRW->"KRW=X", VIX->"VIX", CRUDE->"CL=F", DXY->"DX=F", DJI->"DJI". User-added tickers validated strictly (price > 0); invalid tickers rejected. Edge Function changes require manual redeploy via Supabase Dashboard.
ACTIVE	POST	Fetch Trends	supabase. co/functions/v1/tavily	{ mode: "trends", query: string, max_results?: number, // default 8 include_domains?: string[], time_range?: string }	{ answer: string null, results: [{ title: string, url: string, content: string, published_date: string } null, { score: number null, image: string null }, ...], trends: string[] // article titles for fallback display }	useDataStore.fetchTrends() -> Edge Function tavily/index.ts (mode=trends) -> Tavily Search API (advanced depth)	Auth: no auth required. Default 8 results for trends mode. Korean mode: client-side translateArticlesToKorean() + buildLocalizedTrendTitles() post-processing. Backfills translated payload into api_cache. Cache key: trends_full_(lang).
ACTIVE	POST	Fetch News	supabase. co/functions/v1/tavily	{ mode: "news", query: string, // includes top-5 interest keywords max_results?: number, // default 10 location?: { lat: number, lon: number }, include_domains?: string[] }	{ answer: string null, results: [{ title: string, url: string, content: string, published_date: string } null, { image: string null, score: number null }, ...], location: string null }	useDataStore.fetchNews() -> 2 parallel calls (local + global) -> Edge Function tavily/index.ts (mode=news) -> Tavily News API	Auth: no auth required. Parallel local + global queries, merged max 10 total. Top-5 interest keywords injected into query. Korean mode: translateArticlesToKorean() + filterByAllowedDomains(). Cache key: news_(lang)_(interestFingerprint). Timeout: 25s.
ACTIVE	POST	Tavily Generic Search	supabase. co/functions/v1/tavily	{ mode: "search", query: string, max_results?: number, include_domains?: string[], search_topic?: string, // default "general" time_range?: string }	{ answer: string null, results: [{ title: string, url: string, content: string, published_date: string } null, { image: string null, score: number null }, ...] }	Used internally by aiService.js and smart-widget Edge Function for keyword-specific Tavily searches (shopping, brand pages, restaurant data, etc.)	Auth: no auth required. Same Tavily endpoint as news/trends, parameterized by mode. search_topic defaults to "general" (vs "news" for mode=news). NEW — not in original template.
ACTIVE	POST	Generate Groq Completion	supabase. co/functions/v1/groq	{ prompt: string, // required - user message system?: string, // system message model?: string, // default "llama-3.3-70b-versatile" temperature?: number // default 0.4 }	{ text: string // assistant message content } // Error: { error: string } HTTP 400	aiService.generateBriefing() / generateDetailedBriefing() / aiService.generateDiary() / aiService.rewriteDiaryWithFeedback() / aiService.summarizeArticlesBatch() // up to 6 articles per call Keyword extraction for personalization batch	Auth: no auth required. CORRECTION: Output was { content: STR } — actual is { text: string } (groq/index.ts:46). temperature: 0.1 for factual briefing, 0.4 default. Content-Type: application/json; charset=utf-8 (prevents UTF-8 garbling). Timeout: 15s. Manual redeploy required for Edge Function changes.
ACTIVE	POST	Fetch / Manage Events	supabase. co/functions/v1/events	{ token: string, // Google OAuth provider token action?: "list" "read" "create" "update" "delete", // list: timeMin?: string, timeMax?: string, todayOnly?: boolean, date?: string, // create/update: title?: string, startTime?: string, endTime?: string, location?: string, description?: string, // read/update/delete: eventId?: string }	// list -> array of events: [{ id, title, start, end, status, completed, attendees }] // create/update -> single event object // delete -> { success: true }	useGoogleCalendarStore.fetchEvents() / addEvent() / updateEvent() / deleteEvent() -> token from supabase.auth.getSession(). provider_token -> Edge Function events/index.ts -> Google Calendar API	Auth: Supabase JWT + Google OAuth provider token. Required Google OAuth scope: https://www.googleapis.com/auth/calendar Single endpoint multiplexed by action param. todayOnly: true uses timeMin: now.toISOString() (excludes past). JWT validated server-side via supabase.auth.getUser(jwt) - 401 on failure. Timeout: 25s.
ACTIVE	POST	Fetch / Manage Tasks	supabase. co/functions/v1/tasks	{ token: string, // Google OAuth provider token action?: "list" "create" "update" "delete" "move" "clearCompleted", taskListId?: string, // default "@default" taskId?: string, title?: string, notes?: string, due?: string, completed?: boolean, // move: previousId?: string, parentId?: string }	// list -> array: [{ id, title, notes, due, status, completed, taskListId }] // create/update/move -> single task object // delete/clearCompleted -> { success: true }	useGoogleCalendarStore.fetchTasks() / addTask() / updateTask() / deleteTask() -> token from supabase.auth.getSession(). provider_token -> Edge Function tasks/index.ts -> Google Tasks API	Auth: Supabase JWT + Google OAuth provider token. Required Google OAuth scope: https://www.googleapis.com/auth/tasks Single endpoint multiplexed by action. CORRECTION: Legacy [MB_META]... [MB_META] in notes is stripped on read but no longer written (DOCS.md §5.8). Timeout: 25s.

ACTIVE	POST	Fetch Health	supabase. co/functions/v1/fitness	{ token: string // Google OAuth provider_token }	{ steps: number, sleep: number, // hours calories: number, heartRate: number // bpm (resting) }	useDataStore.fetchHealth() -> token from supabase.auth.getSession() -> Edge Function fitness/index.ts -> Google Fitness aggregate API	Auth: Supabase JWT + Google OAuth provider token. Required scope: https://www.googleapis.com/auth/fitness.activity.read (Restricted scope — annual CASA review required.) Aggregated in 86400s bucket (1 day). Returns zeros on 403 (graceful fallback). Timeout: 25s.
ACTIVE	POST	Smart Widget Query	supabase. co/functions/v1/smart-widget	{ keyword: string, persona?: string, age?: number string, job?: string, interests?: string[], location?: { lat: number, lon: number }, language?: "ko" "en", token?: string // optional Google data enrichment }	{ category: string, sections: [{ type: "summary" "news" "blog" "video", title: string, items: [{ title, url, image?, source?, content? }, ...] }, ...], weather?: { ... } // when category implies outdoor activity }	useWidgetStore.loadSmartWidget() or post Q&A -> Edge Function smart-widget/index.ts -> (1) Groq: category classification + section plan (JSON mode, temp 0.3) -> (2) Tavily search per planned section -> (3) Optional OpenWeatherMap for outdoor context	Auth: Supabase JWT required. CORRECTION: Input was {lat, lon, userInput}. Actual: {keyword, persona, interests, language, ...}. Groq response_format: (type: "json_object"): 17 categories supported (see DOCS.md §7.4). Korean mode: 2-stage Tavily with KO_NEWS_DOMAINS fallback if <2 Korean results. Cache key: \${keyword}_\${lang} (client-side in useWidgetStore).
ACTIVE	GET	Read API Cache	supabase. co/rest/v1/api_cache?id=eq,u:{userId}: {cacheKey}	Query params: id = "u:{userId}: {cacheKey}" e.g. "u:abc-123: weather_37.5_126.9"	{ id: string, data: object, fetched_at: string // ISO timestamp }	useDataStore.readApiCache() -> Supabase REST SELECT -> Check 6h TTL against fetched_at -> CACHE HIT -> get state and return -> CACHE MISS -> proceed to real API call	Auth: Supabase JWT (RLS). Users can only access their own rows. Actual: {userId}: {cacheKey}. Key format: u:{userId}: {cacheKey}. TTL: 360 min / 6h (enforced app-side, not DB). Extended from 60 min on 2026-05-23 (commit 86a34a8) to cut Tavily token consumption. Cache keys are language-aware: news_{lang}_ (interest fingerprint), trends_full_{lang}, \${keyword} \${lang}.
ACTIVE	POST	Write API Cache (UPSERT)	supabase. co/rest/v1/api_cache Header: Prefer: resolution=merge-duplicates	{ id: string, // u:{userId}: {cacheKey} user_id: string, data: object, fetched_at: string // ISO timestamp }	// Empty body on success (HTTP 201)	useDataStore.writeApiCache() -> Supabase REST UPSERT -> Persists after successful API response	Auth: Supabase JWT (RLS). UPSERT via Prefer: resolution=merge-duplicates header. user_id extracted from auth context server-side. Concurrent write safe.
ACTIVE	GET	Get User Settings	supabase. co/rest/v1/user_settings?id=eq,{userId}	Query params: id = {userId}	{ id, persona, theme, tone, briefing_length, voice_on, clock_style, bg_image, vis, stock_symbols, priority_order, show_first_login_briefing, last_briefing_shown, keyword_interests, keyword_interests_updated, fixed_interests, onboarding_perms, pin_lock_mode, created_at, updated_at }	useDataStore.fetchAll() -> Supabase REST SELECT -> Determines visible widgets and user preferences	Auth: Supabase JWT (RLS). Users can only access their own rows. Includes widget visibility, persona, and all preference settings.
ACTIVE	POST	Upsert User Settings	supabase. co/rest/v1/user_settings Header: Prefer: resolution=merge-duplicates	Partial user_settings row (any updatable fields)	// Empty body on success (HTTP 201)	Triggered by all setter actions in useSettingsStore, useAuthStore.finishOB(), etc.	Auth: Supabase JWT (RLS). UPSERT via Prefer: resolution=merge-duplicates. NEW — not in original template.
ACTIVE	GET	Get Widget Layouts	supabase. co/rest/v1/widget_layouts?id=eq,{userId}	Query params: id = {userId}	{ id: string, layouts: { lg: array, md: array, sm: array }, layout_version: number }	useDataStore.fetchAll() -> Supabase REST SELECT -> Restores react-grid-layout positions -> Extracts smart widget IDs from layout entries	Auth: Supabase JWT (RLS). Stores multi-breakpoint grid positions. Smart widget IDs parsed from layout entries.
ACTIVE	POST	Upsert Widget Layouts	supabase. co/rest/v1/widget_layouts Header: Prefer: resolution=merge-duplicates	{ id: string, layouts: { lg: array, md: array, sm: array }, layout_version: number }	// Empty body on success (HTTP 201)	useWidgetStore.handleLayoutChange() / saveDraggedLayout() / resetLayout() -> Supabase REST UPSERT	Auth: Supabase JWT (RLS). NEW — not in original template.
ACTIVE	GET/POST/PATCH/DELETE	Manage Todos	supabase. co/rest/v1/todos	Row shape: { id: uuid, user_id: uuid, text: string, done: boolean, completed: boolean, is_fixed: boolean, is_recurring: boolean // generated from is_fixed }	GET -> array of todo rows POST -> created row PATCH -> updated row DELETE -> empty (HTTP 204)	useTodoStore -> Optimistic local update (temp ID) -> DB upsert -> Temp ID replaced with real UUID on response	Auth: Supabase JWT (RLS). Optimistic UI: temp ID used until DB confirms real UUID. NEW — not in original template.
ACTIVE	GET/POST/DELETE	Manage Smart Keywords	supabase. co/rest/v1/smart_keywords	Row shape: { id: uuid, user_id: uuid, keyword: string } UNIQUE on (user_id, keyword)	GET -> array of keyword rows POST -> created row DELETE -> empty (HTTP 204)	useWidgetStore -> Keyword changes trigger DELETE all + INSERT new (full replace strategy)	Auth: Supabase JWT (RLS). UNIQUE constraint on (user_id, keyword). NEW — not in original template.
ACTIVE	GET/POST/PATCH	Manage Diaries	supabase. co/rest/v1/diaries	Row shape: { user_id: uuid, date: date, // UNIQUE with user_id diary_text: string, memo: string, answers: jsonb, // [{question, answer}] updated_at: timestamptz // auto by trigger }	GET -> array of diary rows POST/PATCH -> upserted row	useDiaryStore.saveDiary() / saveGeneratedDiary() / saveMemo() / confirmRewrite() -> UPSERT on (user_id, date)	Auth: Supabase JWT (RLS). UPSERT key: (user_id, date). updated_at auto-updated by DB trigger. NEW — not in original template.
ACTIVE	GET/POST	Manage Q&A	supabase. co/rest/v1/user_qa	Row shape: { user_id: uuid, question: string, answer: string, asked_date: date } Indexed on (user_id, asked_date)	GET -> array of Q&A rows POST -> inserted row	useDiaryStore.addAnswer() -> INSERT new Q&A row	Auth: Supabase JWT (RLS). Index on (user_id, asked_date) for date-range queries. NEW — not in original template.
ACTIVE	GET/POST	Manage Briefing Snapshots	supabase. co/rest/v1/briefing_snapshots	Row shape: { user_id: uuid, date: date, captured_at: timestamptz, source: "auto" "refresh", payload: jsonb // { text, summary, sections } }	GET -> array of snapshot rows POST -> inserted row	useBriefingHistoryStore.addSnapshot() -> INSERT on 3-hour interval or manual refresh -> INSERT silently no-ops if table doesn't exist	Auth: Supabase JWT (RLS). Optional table — INSERT is wrapped in try/catch; app continues if table absent. NEW — not in original template.

ACTIVE	POST/DELETE	Keyword Score Log	supabase. co/rest/v1/keyword_s core_log	Row shape: { user_id: uuid, keyword: string, category: "food" "place" "content" "shopping" " lifestyle" "mood" " interest", source: "personal" "diary", base_weight: number, // 2=personal Q&A, 1=diary logged_date: date }	POST -> inserted row DELETE -> empty (HTTP 204)	Midnight batch (runPersonalizationBatch) -> INSERT new keyword extractions -> DELETE rows older than 30 days -> Aggregated up to user_settings. keyword_interests	Auth: Supabase JWT (RLS). Retention: 30 days rolling. Aggregated results written to user_settings.keyword_interests. NEW — not in original template.
ACTIVE	—	Sign in with Google (OAuth)	supabase.auth. signInWithOAuth() (Client SDK — no HTTP URL)	{ provider: "google", options: { queryParams: { prompt: "select_account", access_type: "offline" }, scopes: "calendar tasks fitness.activity. read", redirectTo: window.location. origin } }	// Browser redirect — no direct JSON response. // On callback: Supabase exchanges ?code= for session.	Browser redirect to Google consent -> callback with ?code=... -> Supabase exchanges code for session	Auth: N/A (initiates auth flow). prompt: "select_account" avoids forcing full consent every login. access_type: "offline" requests refresh token on first consent. NEW — not in original template.
ACTIVE	—	Get / Refresh Session	supabase.auth. getSession() supabase.auth. refreshSession() (Client SDK — no HTTP URL)	// No direct input — reads Supabase local auth state	session: { access_token: string, // Supabase JWT refresh_token: string, expires_at: number, user: { id, email, user_metadata }, provider_token: string, // Google OAuth token (memory-only) provider_refresh_token: string }	useAuthStore.ensureProviderToken() -> Memory cache first -> getSession() -> refreshSession() if token expired	Auth: N/A (manages auth state). provider_token is NEVER written to localStorage — held only in Zustand memory to prevent XSS token theft. NEW — not in original template.
ACTIVE	—	Sign Out	supabase.auth. signOut() (Client SDK — no HTTP URL)	// No input	// Clears local auth state + revokes server-side session	useAuthStore.logout() -> Clears local Zustand state immediately -> Awaits signOut() to revoke server-side session	Auth: Supabase JWT (sent internally by SDK). NEW — not in original template.